

ARBoids: Adaptive Residual Reinforcement Learning With Boids Model for Cooperative Multi-USV Target Defense

Jiyue Tao[✉], Tongsheng Shen[✉], Dexin Zhao[✉], and Feitian Zhang[✉], *Member, IEEE*

Abstract—The target defense problem (TDP) for unmanned surface vehicles (USVs) concerns intercepting an adversarial USV before it breaches a designated target region, using one or more defending USVs. A particularly challenging scenario arises when the attacker exhibits superior maneuverability compared to the defenders, significantly complicating effective interception. To tackle this challenge, this letter introduces ARBoids, a novel adaptive residual reinforcement learning framework that integrates deep reinforcement learning (DRL) with the biologically inspired, force-based Boids model. Within this framework, the Boids model serves as a computationally efficient baseline policy for multi-agent coordination, while DRL learns a residual policy to adaptively refine and optimize the defenders' actions. The proposed approach is validated in a high-fidelity Gazebo simulation environment, demonstrating superior performance over traditional interception strategies, including pure force-based approaches and vanilla DRL policies. Furthermore, the learned policy exhibits strong adaptability to attackers with diverse maneuverability profiles, highlighting its robustness and generalization capability.

Index Terms—Marine robotics, reinforcement learning, AI-based methods, multi-robot systems.

I. INTRODUCTION

UNMANNED surface vehicles (USVs), with their ability to autonomously navigate and cooperate in complex environments, have garnered significant attention in the marine robotics community [1]. Among diverse applications, the target defense problem (TDP) emerges as a critical and challenging task, wherein one or more defending USVs are deployed to intercept an attacking USV before it breaches a designated target region. This problem holds practical significance in domains such as maritime surveillance, border security, and the protection of marine infrastructure [2], [3], [4].

Received 19 November 2025; accepted 29 January 2026. Date of publication 9 February 2026; date of current version 13 February 2026. This article was recommended for publication by Associate Editor M. Khoramshahi and Editor J. Kober upon evaluation of the reviewers' comments. (*Corresponding authors: Dexin Zhao; Feitian Zhang.*)

Jiyue Tao and Feitian Zhang are with the Robotics and Control Laboratory, School of Advanced Manufacturing and Robotics, State Key Laboratory of Turbulence and Complex Systems, Peking University, Beijing 100871, China (e-mail: jiyuetao@pku.edu.cn; feitian@pku.edu.cn).

Tongsheng Shen and Dexin Zhao are with the National Innovation Institute of Defense Technology, Beijing 100071, China (e-mail: shents_bj@126.com; zhaodx2008@163.com).

The code of this letter is available at <https://github.com/taojy687/ARBoids>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2026.3662620>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2026.3662620

The TDP is inherently complex due to its dynamic and adversarial nature, requiring defenders to make real-time decisions while anticipating the attacker's evasive maneuvers. Prior research has made substantial strides in addressing this problem. For instance, Shishika et al. [5] derived an optimal defensive strategy under partial information constraints. Guo et al. [6] designed a neural network-based controller for cooperative capture. Despite these advances, most existing approaches assume that defenders have comparable or greater agility, thereby limiting their effectiveness in scenarios involving highly maneuverable attackers.

However, scenarios involving attackers with superior maneuverability remain challenging, as conventional strategies often fail against agile evasion. Overcoming this limitation demands a control policy capable of both efficient cooperation and dynamic adaptation to the attacker's real-time behavior. Analytical approaches based on differential game theory and related mathematical frameworks have been extensively explored in recent years [7], [8], [9]. While these methods offer elegant closed-form solutions for idealized adversarial scenarios, they rely on simplifying assumptions such as linearized dynamics and idealized constraints, which do not capture the full complexity of real-world USV systems. Alternatively, force-based methods [10], [11], [12], which employ virtual potential fields to guide robotic movement, has achieved cooperative behaviors. However, they exhibit limited robustness when confronting highly maneuverable adversaries.

Recently, deep reinforcement learning (DRL) has emerged as a powerful tool in multi-agent systems. Prominent algorithms such as MADDPG [13], which extends DDPG with a centralized critic, and QMIX [14], which employs a mixing network to estimate joint action-values, follow the *Centralized Training Decentralized Execution (CTDE)* paradigm. This enables agents to leverage global information during training to develop better coordination strategies while maintaining scalability and robustness through decentralized execution. Consequently, multi-agent RL has gained significant popularity in various applications [15], [16], [17]. However, these DRL-based methods suffer from limitations such as low sample efficiency, poor generalization, and unstable training process. To address these challenges, residual RL has been proposed as a hybrid approach that integrates domain knowledge or heuristic controllers with learned policies. For instance, Johannink et al. [18] formulated the control signal as a superposition of a conventional controller and a residual policy learned via RL, which Rana et al. [19] extended this concept to multi-task motion planning. By leveraging a baseline controller to manage well-understood

aspects of a task, residual RL allows the learned policy to focus on refining actions for unmodeled or complex dynamics. This combination enhances both sample efficiency and robustness, making residual RL a promising solution for the multi-USV TDP problem.

This letter proposes ARBoids, a novel adaptive residual RL framework that integrates DRL with the biologically inspired, force-based Boids model [20], [21]. Specifically, the Boids model serves as a computationally efficient baseline policy for coordination, while DRL learns an adaptive residual policy that refines the defenders' actions. This hybrid approach enables defenders to maintain efficient and scalable behaviors via Boids while dynamically adjusting to complex and challenging scenarios through the learned residual. As a result, ARBoids is particularly effective against attackers with superior maneuverability, combining the robustness of force-based control with the adaptability of learning-based strategies.

We validate the proposed framework through extensive experiments conducted in a high-fidelity Gazebo-based marine simulation environment. Experimental results demonstrate that ARBoids significantly outperforms benchmark strategies — including force-based, standalone DRL, and residual policy approaches — in terms of interception success rate and adaptability. Furthermore, the learned policy generalizes effectively across scenarios with varying numbers of defenders and attackers with different maneuverability levels.

The main contributions of this letter are as follows.

- A novel TDP solution, termed ARBoids, is proposed, integrating DRL with the Boids model to enable cooperative and adaptive defense strategies for USVs.
- An adapter module and its corresponding training mechanism are developed, allowing dynamic adjustment of the weights between DRL and Boids components.
- The proposed ARBoids framework is extensively evaluated in a high-fidelity simulation environment, and its superior performance, generalizability, and robustness are demonstrated.

II. PROBLEM FORMULATION

A. USV Motion Model

In this letter, we employ Virtual RobotX (VRX) [22], a high-fidelity, Gazebo-based marine simulation platform, as the evaluation environment. The WAM-V USV model provided by VRX serves as the deployment robot. Its motion dynamics are based on the model described in [23], expressed as

$$\begin{aligned} M_{RB}\dot{\nu} + C_{RB}(\nu)\nu + M_A\dot{\nu}_r + C_A(\nu_r)\nu_r \\ + D(\nu_r)\nu_r + g(\eta) = \tau + \tau_{\text{wind}} + \tau_{\text{wave}}. \end{aligned} \quad (1)$$

Here, ν and η are the 6-dimensional velocity and position vectors, while ν_r is the vessel velocity relative to the fluid. τ corresponds to the forces and moments generated by USV's left and right propellers. Consistent with [24], the propeller angles are fixed at zero, ensuring that the thrust vectors always align with the longitudinal axis of the hull. Yaw motion is achieved by modulating the thrust differential between two propellers. The maximum forward thrust of a single propeller is denoted as $\tau^{\max}$, while the maximum reverse thrust is represented as $\tau^{\min}$. Consequently, the control input for the USV is defined as the thrust values applied to the two propellers. In this study, external

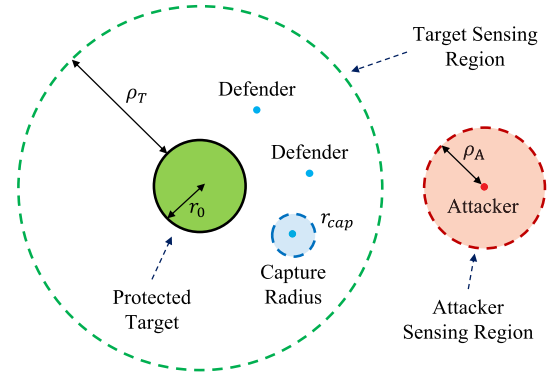


Fig. 1. Schematic diagram illustrating the target defense problem involving multiple defenders and a more agile attacker, as addressed in this letter.

forces such as wind and wave disturbances (τ_{wind} and τ_{wave}) are ignored.

B. USV Target Defense Problem

The multi-USV target defense involving a team of homogeneous USVs tasked with protecting a designated region from a more agile attacking USV, as illustrated in Fig. 1. We adopt a target-centered coordinate frame in which the target is fixed at the origin, i.e., $x_T = (0, 0)$. A trial is considered successful if either of the following conditions is met: (1) the distance between the attacker and at least one defender falls below a predefined capture radius ($d_{i,A} < r_{\text{cap}}$), at any point during the engagement, or (2) the attacker fails to reach the target region within a fixed duration T_{total} . Conversely, a trial is deemed a failure if the attacker breaches the boundary of the circular target region, described by $\|x_A\| \leq r_0$, where x_A represents the attacker's position relative to the target. Additionally, defenders must maintain safe separation throughout the mission. A trial is also classified as a failure if the distance between any two defenders violates the collision threshold ($d_{i,j} \leq r_{\text{collision}}$).

At the initial stage of each trial, the defending USVs are positioned within the protected region, while the attacking USV approaches from a randomly chosen direction within the defenders' sensing range. The propulsion of both the defenders and the attacker is subject to bounded thrust constraints, defined as $\tau_D \in [\tau_D^{\min}, \tau_D^{\max}]$ and $\tau_A \in [\tau_A^{\min}, \tau_A^{\max}]$, respectively. The attacker is granted superior maneuverability, satisfying $\tau_A^{\min} < \tau_D^{\min}$ and $\tau_A^{\max} > \tau_D^{\max}$, thereby enabling greater acceleration, faster maximum speed, and increased maximum angular velocity. For simplicity, we assume a constant ratio between the attacker's and defender's thrust limits, defined as the attacker's agility level, i.e.,

$$\mathcal{L}_{\text{agi}} := \frac{\tau_A^{\min}}{\tau_D^{\min}} = \frac{\tau_A^{\max}}{\tau_D^{\max}}. \quad (2)$$

The multi-USV TDP is formulated as an optimization problem that aims to find a sequence of defender actions $\{\tau_D(t)\}$, which maximizes the probability of task success. This task is subject to the USV dynamics (1), the agility level (2), and additional constraints that govern collision avoidance and thrust limitations. Furthermore, the optimization problem accounts for a fixed adversarial attacker's policy π_A , which governs the attacker's actions based on its observation o_A . The resulting

optimization problem is given by

$$\begin{aligned} \max_{\{\tau_D(t)\}_{t=0}^{T_{\text{total}}}} \quad & \mathbb{P}((d_{i,A} < r_{\text{cap}}, \exists i) \vee (\|\mathbf{x}_A\| > r_0, \forall t)) \\ \text{s.t.} \quad & \tau_D \in [\tau_D^{\min}, \tau_D^{\max}], \quad \tau_A \in [\tau_A^{\min}, \tau_A^{\max}], \\ & \tau_A \sim \pi_A(\mathbf{o}_A), \quad d_{i,j} > r_{\text{collision}}, \quad \forall i \neq j, \\ & \text{USV dynamics (1), and USV agility (2).} \end{aligned} \quad (3)$$

III. DESIGN OF ARBOIDS METHOD

We begin by formulating this problem as a Markov Decision Process (MDP), defined by the 5-tuple $(\mathcal{S}, \mathcal{A}, R, \mathcal{P}, \gamma)$. Here, $\mathcal{S}$ denotes the state space, $\mathcal{A}$ is the action space, R is the reward function, $\mathcal{P}$ is the state-transition probabilities, and γ is the discount factor.

A. Adaptive Residual Policy

The standard residual policy (RP) framework integrates a baseline policy with a learned residual or correction term, enabling the agent to refine and optimize the baseline behavior [18]. Formally, it is defined as

$$\pi_{\text{RP}}(s) = \pi_{\text{DRL}}(s) + \pi_{\text{Baseline}}(s). \quad (4)$$

This structure effectively incorporates the prior knowledge of baseline policy. However, its performance is tightly coupled with the quality of the baseline policy, as the DRL and baseline components contribute equally regardless of context.

In our setting, the Boids model provides a simple yet limited defensive heuristic, which alone is inadequate for handling the complex and dynamic nature of the TDP. To address this limitation, we propose the ARBoids framework, an adaptive residual policy built upon the Boids model. This approach introduces a learnable and state-dependent weighting parameter, $\theta(s)$, which dynamically adjust the relative contributions of the DRL policy and the Boids policy, i.e.,

$$\pi_{\text{ARBoids}}(s) = \theta(s) \cdot \pi_{\text{DRL}}(s) + (1 - \theta(s)) \cdot \pi_{\text{Boids}}(s), \quad (5)$$

where $\theta(s) \in [0, 1]$ is dynamically computed to balance the contributions of π_{DRL} and π_{Boids} . Notably, when $\theta(s) = 0$, the policy falls back entirely on the Boids model, and when $\theta(s) = 1$, it relies solely on the DRL policy. We train $\theta(s)$ end-to-end within our DRL framework to maximize expected discounted return. $\theta^*(s)$ represents the optimal state-dependent parameter, defined as

$$\theta^*(s) = \arg \max_{\theta \in [0,1]} Q^{\pi_{\text{ARBoids}}}(s, \pi_{\text{ARBoids}}(s)), \quad (6)$$

where $Q^{\pi}(s, a)$ represents the expected discounted return starting from the state-action pair (s, a) following policy π , i.e.,

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s, a_0 = a \right]. \quad (7)$$

Here, r_{t+1} denotes the reward at time step $t + 1$. In practice, $\theta(s)$ is generated by an adapter network, which is trained jointly with the actor-critic architecture. This end-to-end optimization enables the framework to focus on maximizing cumulative rewards, thereby improving long-term performance.

B. Soft Actor Critic

We employ the soft actor-critic (SAC) algorithm [25], a state-of-the-art method known for its robustness and sample efficiency. SAC incorporates an entropy-maximization framework, optimizing the following objective

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))], \quad (8)$$

where $\mathcal{H}(\pi(\cdot | s_t))$ denotes the entropy of the policy π , and α is a temperature parameter balancing exploration and exploitation. This formulation encourages diverse action selection, enhances robustness to dynamic uncertainties, and accelerates policy convergence. Furthermore, our learning framework follows the CTDE paradigm, combined with parameter sharing. A single shared policy is trained using experiences aggregated from all homogeneous defenders, facilitating efficient data utilization and accelerated convergence. During execution, each agent selects actions autonomously based on its own local observations, yielding a fully decentralized control strategy.

C. Boids Model

The Boids model, originally proposed by C. W. Reynolds [20], is a biologically inspired computational framework for simulating the collective behavior of swarm-like agents, such as bird flocks or fish schools. This decentralized model governs the behavior of each agent using three fundamental rules — separation, alignment, and cohesion. To enable pursuit of the attacker in our context, we extend the standard Boids model by introducing an additional attraction rule. The mathematical formulations employed in this letter are defined as follows

- **Separation** ($\mathbf{F}_{\text{sep}}$): Prevents collisions by enforcing a repulsive force that maintains a safe distance from neighboring agents.

$$\mathbf{F}_{i,\text{sep}} = -k_{\text{sep}} \sum_{j \in \mathcal{N}_i} \frac{\mathbf{x}_j - \mathbf{x}_i}{\|\mathbf{x}_j - \mathbf{x}_i\|}, \quad (9)$$

where k_{sep} is the separation weight, $\mathbf{x}_j$ is the position of neighbor j , and $\mathcal{N}_i$ is the set of neighbors of agent i .

- **Alignment** ($\mathbf{F}_{\text{ali}}$): Steers the agent to align its velocity with the average velocity of its neighbors.

$$\mathbf{F}_{i,\text{ali}} = \frac{k_{\text{ali}}}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{v}_j - \mathbf{v}_i, \quad (10)$$

where k_{ali} is the alignment weight, and $\mathbf{v}_j$ is the velocity of neighbor j .

- **Cohesion** ($\mathbf{F}_{\text{coh}}$): Encourages movement toward the centroid of neighboring agents to maintain group coherence.

$$\mathbf{F}_{i,\text{coh}} = \frac{k_{\text{coh}}}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{x}_j - \mathbf{x}_i, \quad (11)$$

where k_{coh} is the cohesion weight.

- **Attraction** ($\mathbf{F}_{\text{att}}$): Guides agents toward the attacker's current position to facilitate interception.

$$\mathbf{F}_{i,\text{att}} = k_{\text{att}}(\mathbf{x}_A - \mathbf{x}_i), \quad (12)$$

where k_{att} is the attraction weight.

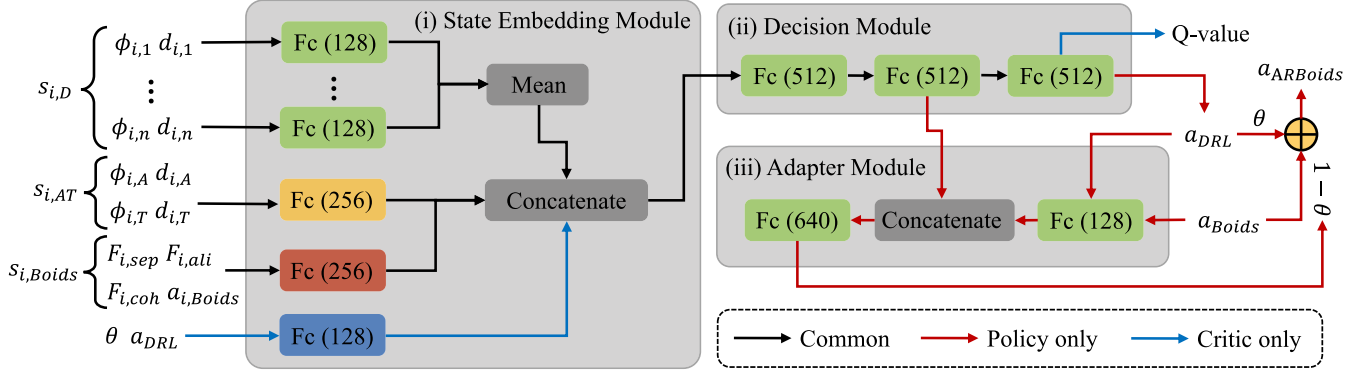


Fig. 2. Architecture of the policy and critic networks for a single defender agent. The policy network comprises three modules: (i) State embedding, which processes the state component $s_{i,AT}$ and $s_{i,Boids}$ independently via fully-connected (Fc) layers, and encodes $s_{i,D}$ using mean observation embedding [26]; (ii) Decision, which applies three Fc layers to the concatenated feature vector to produce the DRL action a_{DRL} ; and (iii) Adapter, which integrates a_{DRL} , the Boids action a_{Boids} , and the decision-module hidden features. The adapter outputs a weighting coefficient θ , linearly mapped to $[0,1]$, to adaptively weight a_{DRL} and a_{Boids} . The critic network uses modules (i) and (ii), and includes an additional action-embedding layer to encode a_{DRL} and θ .

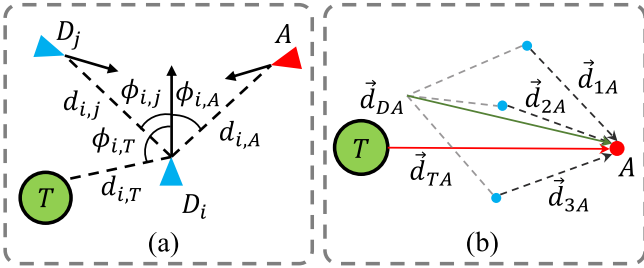


Fig. 3. (a) State space components $s_{i,D}$ and $s_{i,AT}$ for each defender i , encompassing information about other defenders (D), the attacker (A), and the target region (T). All observations are expressed in local coordinates. (b) Diagram of components included in the formation reward r_{form} . Here, $\vec{d}_{iA}$ denotes the unit vector from defender i to the attacker, $\vec{d}_{TA}$ is the unit vector from the target to the attacker, and $\vec{d}_{DA} = \sum_{i=1}^n \vec{d}_{iA}$ is the cumulative direction vector of all defenders.

The total virtual force acting on defender i is the sum of all individual components, i.e.,

$$\mathbf{F}_i = \mathbf{F}_{i,sep} + \mathbf{F}_{i,ali} + \mathbf{F}_{i,coh} + \mathbf{F}_{i,att}. \quad (13)$$

The virtual force $\mathbf{F}_i$ is subsequently transformed into the vehicle's local coordinate frame, and then linearly mapped to generate the thrust commands, which collectively define the Boids action $a_{i,Boids}$ of defender i . The Boids hyperparameters are initialized based on established guidelines from the literature and fine-tuned in our simulator to provide a stable baseline policy.

D. State Representation

As shown in Fig. 2, the state space of defender i — assuming a total of n defenders — is composed of three components and is defined as $s_i = [s_{i,D}, s_{i,AT}, s_{i,Boids}]$. The first component, $s_{i,D} = [\phi_{i,1}, d_{i,1}, \dots, \phi_{i,n}, d_{i,n}]$, captures the relative bearing and distance observations of other defenders, as illustrated in Fig. 3(a). Its dimensionality scales with the number of defenders. The second component, $s_{i,AT} = [\phi_{i,A}, d_{i,A}, \phi_{i,T}, d_{i,T}]$, encodes the relative positions of the attacker and the target. To better approximate real-world sensing conditions, we incorporate observation noise into this component during training.

The third component, $s_{i,Boids} = [\mathbf{F}_{i,sep}, \mathbf{F}_{i,ali}, \mathbf{F}_{i,coh}, a_{i,Boids}]$, contains the virtual forces defined in (13) and the resulting action $a_{i,Boids}$ generated by the Boids model. This component serves as the baseline policy and implicitly encodes information regarding teammate positions and group formation, thereby assisting the DRL agent in learning a more effective residual policy.

E. Network Structure

The proposed neural network structure consists of three main modules: state embedding module, decision module, and adapter module. As illustrated in Fig. 2, the state components $s_{i,AT}$ and $s_{i,Boids}$ are processed independently through single fully-connected (Fc) layers, each with a specific number of hidden units. Meanwhile, $s_{i,D}$ is encoded using a mean observation embedding strategy, following the approach outlined in [26]. Specifically, each observation related to a teammate is passed through an Fc layer, and the resulting features are averaged to produce the final embedding. The Leaky ReLU is selected as the activation function. The three resulting feature vectors are then concatenated and input into the subsequent layers of the network. The critic network employs a similar architecture but includes an additional action embedding layer to process the action input.

As shown in Fig. 2, the decision module is composed of three Fc layers that process the feature vector produced by the embedding network to generate the DRL action a_{DRL} . The adapter module integrates the DRL action, the Boids action, and the hidden representation from the decision module. This combined encoded vector is subsequently passed through a Tanh-activated layer and linearly mapped to the interval $[0, 1]$, ensuring that the weighting coefficient θ remains a valid proportion. This architecture enables a dynamic adjustment mechanism that adaptively balances the influence of the learned DRL policy and the baseline Boids model.

A key challenge in learning an additional adapter module lies in ensuring efficient exploration. Given the off-policy nature of the SAC algorithm, an exploration policy $\theta'(s)$ is introduced by injecting stochastic noise into the output of the adapter during training, defined as

$$\theta'(s) = \text{clip}(\theta(s) + \mathcal{N}(0, 0.1), 0, 1), \quad (14)$$

where $\mathcal{N}(a, b)$ represents a Gaussian distribution with mean a and standard deviation b , and $\text{clip}(x, a, b)$ constrains x within the interval $[a, b]$. This stochastic exploration strategy facilitates the discovery of improved blending policies by the adapter module. Additionally, the adaptive weight θ is explicitly incorporated as an input to the critic network, alongside the DRL action a_{DRL} , to provide gradient updates for the adapter module.

F. Reward Function

In the proposed framework, each agent receives its own reward at every time step, designed as

$$r_i = r_{\text{main}} + r_{\text{form}} + r_{\text{collision}}, \quad (15)$$

The main reward r_{main} provides feedback on task success or failure and is defined as

$$r_{\text{main}} = \begin{cases} -100, & \text{if } \|\mathbf{x}_A\| \leq r_0, \\ 100, & \text{if } d_{i,A} \leq r_{\text{cap}}, \\ 50, & \text{if } d_{i,A} \leq 3 \cdot r_{\text{cap}} \text{ s.t. } d_{j,A} \leq r_{\text{cap}}, \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

Specifically, a penalty of -100 is assigned if the attacker reaches the target region, while a reward of 100 is granted upon successful capture by the agent. A helper reward of 50 is provided if a teammate captures the attacker and the agent is within $3 \cdot r_{\text{cap}}$ of the capture location. This design promotes collaboration among defenders.

The formation reward r_{form} encourages defenders to maintain an effective defensive formation around the attacker. As illustrated in Fig. 3(b), $\vec{d}_{iA}$ represents the unit vector pointing from defender i toward the attacker, and $\vec{d}_{DA} = \sum_{i=1}^n \vec{d}_{iA}$ represents the cumulative direction vector of all defenders. The formation reward is computed as

$$r_{\text{form}} = 0.5 \cdot \vec{d}_{TA} \cdot \frac{\vec{d}_{DA}}{\|\vec{d}_{DA}\|} - \frac{\|\vec{d}_{DA}\|}{n}, \quad (17)$$

where $\vec{d}_{TA}$ is the unit vector from the target to the attacker. The first term promotes a symmetric distribution of defenders along the direction of threat, while the second term incentivizes spatial dispersion to maximize coverage.

To discourage interference among defenders, the collision penalty is defined as $r_{\text{collision}} = -50$ if a collision occurs ($d_{i,j} \leq r_{\text{collision}}$), and $r_{\text{collision}} = 0$ otherwise. No additional inter-agent attraction or spacing rewards are applied, as such mechanisms are inherently similar to the Boids virtual forces, leading to redundancy. Besides, the reward components are carefully scaled to reflect their relative importance. Specifically, r_{main} and $r_{\text{collision}}$ are prioritized to dominate the optimization process, while r_{form} serves as auxiliary guidance.

G. Curriculum Learning

Curriculum learning [27] is employed to enhance training stability and mitigate the risk of converging to suboptimal local minima. In this work, we progressively increase the TDP task difficulty to facilitate effective exploration during the early stages of training. The primary factor influencing task difficulty is the attacker's agility level, $\mathcal{L}_{\text{agi}}$. At the beginning of each training episode, $\mathcal{L}_{\text{agi}}$ is randomly sampled from a uniform distribution as follows,

$$\mathcal{L}_{\text{agi}} \sim \mathcal{U}(\bar{\mathcal{L}}_{\text{agi}} - 0.5, \bar{\mathcal{L}}_{\text{agi}} + 0.5), \quad (18)$$

where $\bar{\mathcal{L}}_{\text{agi}}$ represents the average agility level. To initiate learning with manageable complexity, $\mathcal{L}_{\text{agi}}$ is initially set to a relatively low value ($\bar{\mathcal{L}}_{\text{agi}} = 2.0$), enabling agents to effectively learn the fundamental dynamics of the task. Over the course of training, $\bar{\mathcal{L}}_{\text{agi}}$ is increased by 0.25 every 2.5×10^5 training steps, which corresponds to one-quarter of the total training duration. This progressive increase in task difficulty encourages the agents to develop robust and generalizable defensive strategies, rather than overfitting to simplified scenarios with limited attacker agility.

IV. EXPERIMENTS AND ANALYSIS

A. Experimental Setup

1) *Experiment Parameters*: The simulation parameters are configured as follows. The capture radius r_{cap} is set to 5 m. The target radius r_0 , the target's sensing range ρ_T , and the attacker's sensing range ρ_A are set to 15 m, 60 m, and 15 m, respectively. The maximum forward thrust of each defending USV τ_D^{max} is set to 1000 N, while the maximum reverse thrust τ_D^{min} is set to -500 N. The total simulation time is $T_{\text{total}} = 60$ s, and the action interval is $T_{\text{action}} = 0.2$ s. The Boids model weights are configured with separation $k_{\text{sep}} = 10$, alignment $k_{\text{ali}} = 0.1$, cohesion $k_{\text{coh}} = 0.1$, and attraction $k_{\text{att}} = 0.5$. During training, the number of defenders is fixed at 3 . The SAC algorithm is adopted with a learning rate of 1×10^{-4} , a batch size of 4096 , a total of 1×10^6 training steps, and a discount factor of 0.99 .

2) *Attacker's Policy*: The attacker employs an improved artificial potential field (APF) strategy [28], which combines both attractive and repulsive forces. The target region generates an attractive force that pulls the attacker toward the target, while each defender within the attacker's sensing range induces a repulsive force to steer it away. Furthermore, we investigate the robustness of our approach against learning-based attacker policies. An alternating learning framework is adopted, in which the attacker and defender alternately learn their respective policies. During each phase, one side's policy is fixed while the other undergoes training, enabling iterative mutual adaptation and performance improvement. The attacker's policy is optimized using the SAC algorithm, guided by a deliberately designed reward function that balances the objectives of reaching the target and avoiding defenders.

3) *Training Environment*: To enable large-scale DRL roll-outs with stable wall-clock time, the training environment employs a custom numerical simulator based on the planar 3-DoF dynamic model. This model simplifies the vessel's dynamics (1) to the horizontal plane, focusing on surge, sway, and yaw rate components, which retains the key maneuvering dynamics necessary for the tasks. The environment and models are implemented in Python, with PyTorch used for neural network development. Training and evaluations are performed on a workstation equipped with an Intel i7-14650HX CPU and an NVIDIA GeForce RTX 4070Ti GPU. Under this setup, running 1 million training steps with our method requires approximately 7.6 hours of wall-clock time.

B. Experimental Results

As illustrated in Fig. 4, the proposed ARBoids framework is evaluated within the Gazebo VRX simulation using the `sydney_regatta` environment. The experiment simulates an

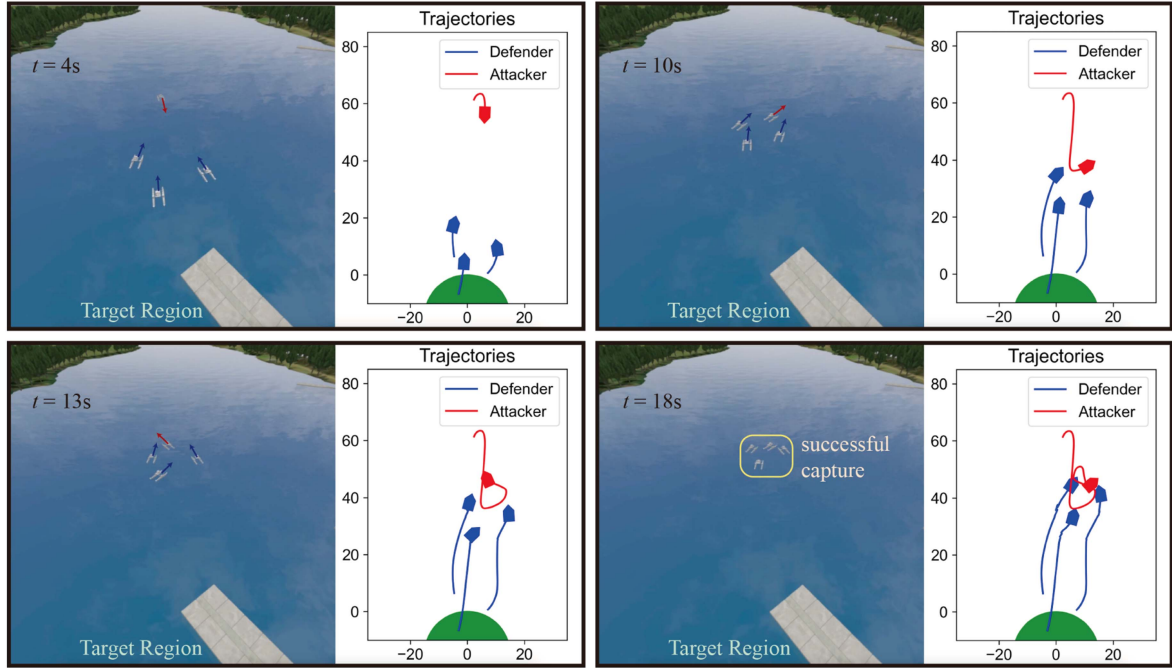


Fig. 4. Snapshots (left) and trajectories (right) from a representative Gazebo VRX experiment deploying the proposed ARBoids method. VRX is a high-fidelity marine simulator and the USV model follows the dynamics in (1). The attacker, with an agility level of $\mathcal{L}_{\text{agi}} = 2.25$, tries to penetrate the dock area. The timestamp t is displayed in the upper-left corner of each snapshot. Red and blue arrows denote the heading directions of the attacker and defender, respectively.

adversarial scenario where an attacking USV, with an agility level of $\mathcal{L}_{\text{agi}} = 2.25$, attempts to navigate along the river to reach the dock. To counter this threat, three defending USVs are deployed from the dock, coordinating their efforts to intercept the attacker. Each subfigure depicts a simulation snapshot and the corresponding trajectories at a specific timestamp t .

At the early stage ($t = 4$ s), the attacker advances at full speed toward the target region, while the defenders initiate coordinated interception. By $t = 13$ s, the attacker evades the initial attempted capture and redirect its trajectory to approach the dock from a different angle. Rather than engaging in direct pursuit, the defenders predict the attacker's path and reposition to block its progress, forcing another retreat. After several such maneuvers, the defenders successfully capture the attacker at $t = 18$ s. Throughout the process, the defenders maintain a cohesive formation, dynamically adjusting their positions to maximize spatial coverage while avoiding collisions. These results demonstrate that ARBoids enables intelligent, strategic, and cooperative defensive behaviors that adapt in real time to the evolving situation.

A total of 100 trials are conducted, with the attacker's initial positions uniformly sampled from a predefined angular sector, while all other conditions remain unchanged. The defenders achieve a success rate (SR) of 85%, with all successful outcomes resulting in the attacker being captured. Among the 15 failed trials, 11 are caused by inter-defender collisions that compromise the agents' ability to complete the interception, while the remaining 4 involve the attacker successfully breaching the defense to reach the dock. These findings highlight the robustness of the ARBoids framework across diverse initial conditions. Furthermore, they suggest that future enhancements in collision avoidance mechanisms may yield additional performance gains.

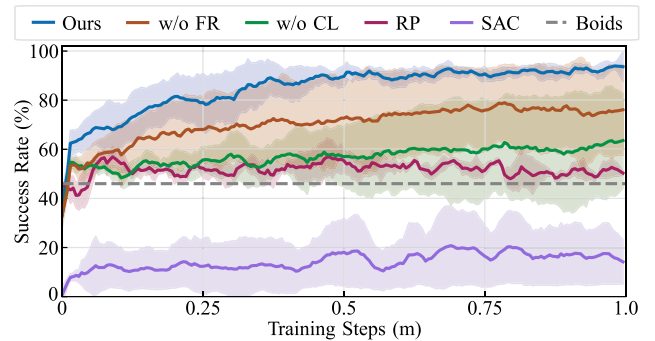


Fig. 5. Learning curves comparing our full method (ARBoids), ablations (w/o formation reward, FR; w/o curriculum learning, CL), and baselines (RP, SAC, Boids). Success rates are measured every 5,000 steps to assess learning progress. Shaded regions indicate the standard deviation across 5 independent runs. ARBoids substantially outperforms all baselines, and removing either FR or CL results in a pronounced degradation in performance.

C. Comparison With Benchmark Methods

1) *Benchmark Methods*: To demonstrate the advantages of our proposed method, we quantitatively evaluate its performance by comparing its success rate (SR) and convergence speed against three benchmark approaches: the Boids model, the standard RP, and a vanilla DRL policy trained using the SAC algorithm. For the Boids model, all weights are configured identically to those used in our method to ensure a fair comparison. In the standard residual policy, action commands are computed as described in (4). For the vanilla SAC policy, the Boids-related state component $s_{i,\text{Boids}}$ is excluded from the state space. Each method is evaluated for 100 trials under identical experimental conditions.

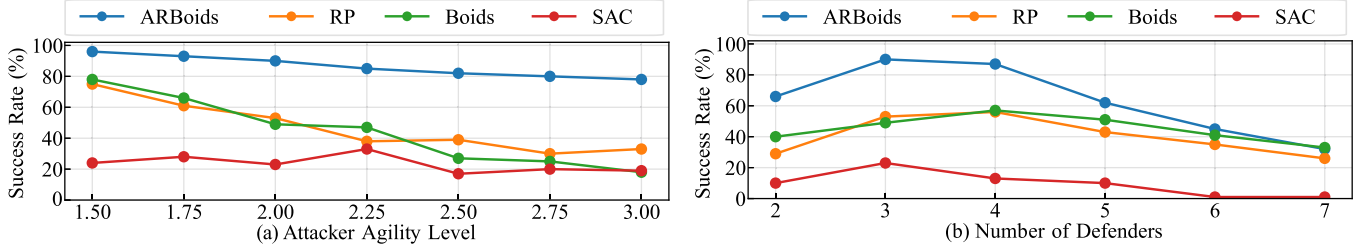


Fig. 6. Statistical comparison of success rates (SRs) against benchmark methods. (a) SRs under varying levels of attacker agility, evaluated with a fixed number of defenders ($n = 3$). (b) SRs as a function of the number of defenders, with the attacker's agility level fixed at $\mathcal{L}_{\text{agi}} = 2.0$. These results highlight the adaptability and robustness of the proposed method across different scenario parameters.

2) *Comparison Results:* We track the SRs of three learning-based methods—SAC, RP and ARBoids along with two ablations of ARBoids without the formation reward (w/o FR) and without curriculum learning (w/o CL). Each method is assessed every 5,000 steps under a fixed attacker agility level of $\mathcal{L}_{\text{agi}} = 2.0$. As shown in Fig. 5, vanilla SAC fails to converge, with SR stagnating near 20% even after 1 million steps. Both RP and ARBoids benefit from the Boids prior, yielding a strong initial SR of about 45% by promoting exploration via more frequent positive rewards. However, RP quickly plateaus around 55% after 0.1 million steps, indicating limited capacity for further improvement. By contrast, ARBoids converges rapidly and reliably, reaching 90% SR within 0.35 million steps—an absolute gain of 35% over RP and 70% over SAC. The ablations confirm the contributions of both components. Removing either FR or CL leads to a significant performance degradation. Overall, ARBoids achieves superior sample efficiency, stability, and final SR, validating the effectiveness of its adaptive hybrid design.

D. Robustness Against Attacker Agility Levels

We evaluate the SRs of our method under varying attacker agility levels, with $n = 3$ defenders. As illustrated in Fig. 6(a), the SRs of all methods decline as the attacker's agility increases, reflecting the growing difficulty of maintaining effective defense under more agile adversarial behavior. Across all agility levels, the proposed ARBoids consistently and significantly outperforms the comparison methods, with the performance gap widening as the problem becomes more challenging. Specifically, while the Boids model and the standard RP achieve SRs close to 80% at $\mathcal{L}_{\text{agi}} = 1.5$, their performance deteriorates sharply to below 40% when $\mathcal{L}_{\text{agi}} \geq 2.5$. In contrast, ARBoids exhibits a relatively modest performance decrease of approximately 18%, maintaining high SRs even under challenging conditions. These findings underscore the robustness and generalizability of ARBoids across a wide range of attacker agility levels—an essential capability for real-world deployment scenarios where the attacker's agility is unknown or variable.

E. Generalization to Unseen Team Size

We evaluate the generalization capability of ARBoids under unseen defender team sizes. Although the policy is exclusively trained with three defenders, it successfully generalizes across different team sizes (n), enabled by the mean observation embedding technique [26]. The corresponding SRs are illustrated in Fig. 6(b). As the number of defenders increases, the defense

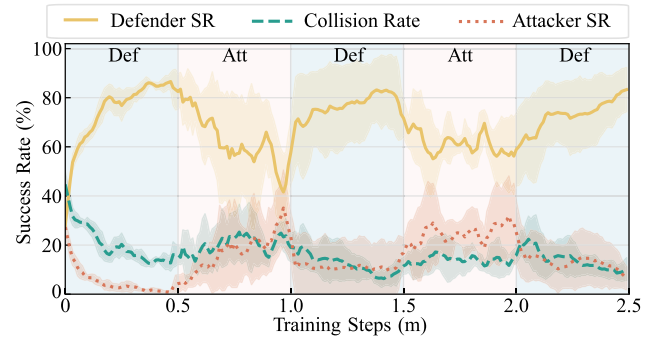


Fig. 7. Learning curves of the defender and attacker during alternating training. The curves illustrate the defender success rate (Defender SR), collision rate, and attacker success rate (Attacker SR) as functions of training steps. The blue-shaded regions indicate the defender's learning phases, whereas the red-shaded regions correspond to the attacker's learning phases.

scenario becomes more complex. While additional defenders expand coverage and interception capability, they also introduce challenges such as increased collision risk and higher coordination demands. ARBoids maintains high SRs for $n = 3$ and $n = 4$, demonstrating robustness in typical deployment scenarios. However, performance begins to degrade when $n > 4$, indicating that densely coordinated environments require stronger coordination and collision avoidance mechanisms, which the current policy, trained only with three defenders, cannot fully provide. To address this, we plan to explicitly train ARBoids on larger defender teams and enhance its scalability through techniques such as value decomposition networks and behavior prediction for collision avoidance.

F. Robustness Against Learning-Based Attacker

We further evaluate the robustness of ARBoids against a learning-based attacker by training the defender and attacker alternately over five phases, totaling 2.5 million training steps. The resulting learning dynamics of both agents during this process are shown in Fig. 7. During each attacker learning phase, the defender's SR temporarily decreases as the attacker learns to exploit weaknesses in the defender's policy. However, it quickly recovers and improves in the subsequent defender learning phase, demonstrating the effectiveness of our method in adapting to evolving attacker strategies through fine-tuning. Additionally, the inter-vehicle collision rate of defenders demonstrates a steady decline from an initially high value. This suggests that defenders not only develop policy-specific counter-strategies but

also acquire general collision-avoidance capabilities, highlighting their improved adaptability. It is worth noting, however, that the final defender policy derived from this framework exhibits a slight performance degradation when tested against the original APF-based attacker. This indicates a potential trade-off, where the defender tends to overfit to the specific strategies of the learning-based adversary encountered during training.

V. CONCLUSION

This letter investigated the target defense problem involving an agile attacking USV and a team of homogeneous defending USVs. We proposed a novel defending control strategy, ARBoids, which integrated the SAC algorithm with the biologically inspired Boids model. The Boids model served as a baseline coordination policy, while an SAC agent learned a residual policy to optimize defenders' actions. An adapter module dynamically balanced these contributions. A tailored reward function and curriculum learning enhanced training stability and generalization. Extensive experiments conducted in a high-fidelity marine simulation environment demonstrated that ARBoids significantly outperformed selected benchmark methods. Future work will enhance the generalization of ARBoids across diverse and unseen attacker strategies by further exploring the alternating training with techniques such as adversarial policy ensembles. We also plan to develop advanced collision avoidance techniques, investigate scaling limits, and validate our approach in real-world experiments.

REFERENCES

- [1] Y. Qiao, J. Yin, W. Wang, F. Duarte, J. Yang, and C. Ratti, "Survey of deep learning for autonomous surface vehicles in marine environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3678–3701, Apr. 2023.
- [2] Y. Chen, Y. Wu, H. Yang, J. Cao, Q. Wang, and Y. Liu, "A distributed pipeline for collaborative pursuit in the target guarding problem," *IEEE Robot. Automat. Lett.*, vol. 9, no. 3, pp. 2064–2071, Mar. 2024.
- [3] X. Fu and Y. Sun, "A combined intrusion strategy based on apollonius circle for multiple mobile robots in attack-defense scenario," *IEEE Robot. Automat. Lett.*, vol. 10, no. 1, pp. 676–683, Jan. 2025.
- [4] H. Fu and H. H.-T. Liu, "Guarding a territory against an intelligent intruder: Strategy design and experimental verification," *IEEE/ASME Trans. Mechatron.*, vol. 25, no. 4, pp. 1765–1772, Aug. 2020.
- [5] D. Shishika, D. Maity, and M. Dorothy, "Partial information target defense game," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 8111–8117.
- [6] X. Guo, R. Cui, and W. Yan, "Pursuit-evasion games of marine surface vessels using neural network-based control," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 1, pp. 18–27, Jan. 2025.
- [7] R. Yan, X. Duan, R. Zou, X. He, Z. Shi, and F. Bullo, "Multiplayer homicidal chauffeur reach-avoid games: A pursuit enclosure function approach," *Automatica*, vol. 167, 2024, Art. no. 111770.
- [8] E. Garcia, D. W. Casbeer, and M. Pachter, "Optimal strategies for a class of multi-player reach-avoid differential games in 3D space," *IEEE Robot. Automat. Lett.*, vol. 5, no. 3, pp. 4257–4264, Jul. 2020.
- [9] Y. Lee and E. Bakolas, "Guarding a convex target set from an attacker in Euclidean spaces," *IEEE Control Syst. Lett.*, vol. 6, pp. 1706–1711, 2021.
- [10] M. Janosov, C. Virágh, G. Vásárhelyi, and T. Vicsek, "Group chasing tactics: How to catch a faster prey," *New J. Phys.*, vol. 19, no. 5, 2017, Art. no. 053003.
- [11] R. Escobedo, C. Muro, L. Spector, and R. Coppinger, "Group size, individual role differentiation and effectiveness of cooperation in a homogeneous group of hunters," *J. Roy. Soc. Interface*, vol. 11, no. 95, 2014, Art. no. 20140204.
- [12] X. Fang, C. Wang, L. Xie, and J. Chen, "Cooperative pursuit with multi-pursuer and one faster free-moving evader," *IEEE Trans. Cybern.*, vol. 52, no. 3, pp. 1405–1414, Mar. 2022.
- [13] R. Lowe et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6382–6393.
- [14] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 21, no. 178, pp. 1–51, 2020.
- [15] C. De Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulić, "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 4552–4559, Jul. 2021.
- [16] Z. Zhang, X. Wang, Q. Zhang, and T. Hu, "Multi-robot cooperative pursuit via potential field-enhanced reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2022, pp. 8808–8814.
- [17] W. Jin, X. Tian, B. Shi, B. Zhao, H. Duan, and H. Wu, "Enhanced UAV pursuit-evasion using boids modelling: A synergistic integration of bird swarm intelligence and DRL," *Comput. Mater. Continua*, vol. 80, no. 3, pp. 3523–3553, 2024.
- [18] T. Johannink et al., "Residual reinforcement learning for robot control," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2019, pp. 6023–6029.
- [19] K. Rana, M. Xu, B. Tidd, M. Milford, and N. Sünderhauf, "Residual skill policies: Learning an adaptable skill-based action space for reinforcement learning for robotics," in *Proc. Conf. Robot. Learn.*, 2023, pp. 2095–2104.
- [20] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," *Comput. Graph.*, vol. 21, no. 4, pp. 25–34, 1987.
- [21] I. L. Bajec, N. Zimic, and M. Mraz, "The computational beauty of flocking: Boids revisited," *Math. Comput. Model. Dyn. Syst.*, vol. 13, no. 4, pp. 331–347, 2007.
- [22] B. Bingham et al., "Toward maritime robotic simulation in Gazebo," in *Proc. MTS/IEEE OCEANS Conf.*, 2019, pp. 1–10.
- [23] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. Hoboken, NJ, USA: Wiley, 2011.
- [24] X. Lin, P. Szenher, Y. Huang, and B. Englot, "Distributional reinforcement learning based integrated decision making and control for autonomous surface vehicles," *IEEE Robot. Automat. Lett.*, vol. 10, no. 2, pp. 1194–1201, Feb. 2025.
- [25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [26] M. Hüttenrauch, A. Šošić, and G. Neumann, "Deep reinforcement learning for swarm systems," *J. Mach. Learn. Res.*, vol. 20, no. 54, pp. 1–31, 2019.
- [27] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. Int. Conf. Mach. Learn.*, 2009, pp. 41–48.
- [28] Y. Zhang, J. Chen, M. Chen, C. Chen, Z. Zhang, and X. Deng, "Integrated the artificial potential field with the leader-follower approach for unmanned aerial vehicles cooperative obstacle avoidance," *Mathematics*, vol. 12, no. 7, 2024, Art. no. 954.